Vidocq

← Blog · oct. 15, 2026

# Vaadin on Vidocq: from vidocq:dev to a 68 MB jlink Image

Vaadin Flow now runs on Vidocq and Foy, our Servlet 6.1 container, with no workaround class: in development, as a packaged distribution, and as a self-contained jlink image. Here is how to set it up, what each way of running it gives you, and what we had to fix on the way, since a framework built for the class path met a runtime built for Java modules.

# vaadin # servlet # java-modules # jlink # deep-dive




Vaadin is the framework many Java teams reach for when they want a web UI without writing JavaScript: you write components in Java, and Vaadin keeps the browser in sync. It runs on any servlet container. Vidocq now has one of its own, Foy, which implements Servlet 6.1. So the question was obvious: does Vaadin run on Vidocq?

It does, and without a single line of glue in your application. It runs under `vidocq:dev` and `vidocq:run`, ships as a `vidocq:package` distribution, and links into a `vidocq:jlink` image of 68 MB that brings its own JDK, where the runtime is ready in

about a third of a second. This post shows how to set it up and what each way of running it gives you. It also tells the story behind it: Vaadin was built for the class path, Vidocq for Java modules, and getting the two to agree took more fixes than we expected.

## The setup

A Vaadin application on Vidocq is an ordinary Vidocq application, with `vidocq-runtime-parent` as its parent, and four more pieces: the Servlet extension, the Vaadin extension, the Vaadin BOM (`com.vaadin:vaadin-bom`, imported in `dependencyManagement`), and Vaadin's Maven plugin to build the frontend. Here is the core of the pom:

```
<properties>
  <!-- The application's Java module: the runtime's own main boots it, n
  <vidocq.mainModule>com.example.app</vidocq.mainModule>
  <vidocq.app.layer>all</vidocq.app.layer>
  <!-- Vaadin's production mode under vidocq:dev, vidocq:run and in the
  <vidocq.dev.jvmArgs>-Dvaadin.productionMode=true</vidocq.dev.jvmArgs>
  <vidocq.run.jvmArgs>-Dvaadin.productionMode=true</vidocq.run.jvmArgs>
  <vidocq.jvmArgs>-Dvaadin.productionMode=true</vidocq.jvmArgs>
</properties>

<dependencies>
  <dependency>
    <groupId>io.vidocq.runtime</groupId>
    <artifactId>vidocq-runtime-core</artifactId>
  </dependency>
  <dependency>
    <groupId>io.vidocq.runtime.extensions.jakartaee.web</groupId>
    <artifactId>vidocq-runtime-foy-servlet-extension</artifactId>
  </dependency>
  <!-- Lets Vaadin find your AppShellConfigurator (theme, stylesheets) i
  <dependency>
    <groupId>io.vidocq.runtime.extensions.jakartaee.web</groupId>
    <artifactId>vidocq-runtime-vaadin-extension</artifactId>
  </dependency>
  <dependency>
    <groupId>com.vaadin</groupId>
    <artifactId>vaadin-core</artifactId>
  </dependency>
```

```
    <!-- The precompiled frontend of the core components: production mode
    <dependency>
      <groupId>com.vaadin</groupId>
      <artifactId>vaadin-prod-bundle</artifactId>
      <version>${vaadin.version}</version>
    </dependency>
  </dependencies>
```

On top of that, two plugins run: `vaadin-maven-plugin` with `prepare-frontend` and `build-frontend`, and `vidocq-runtime-maven-plugin` with `generate`. The parent manages every Vidocq version, and the Vidocq plugin puts Foy's annotation processors on the compiler's path by itself: an extension is one dependency, with no `annotationProcessorPaths` to write. There is no main class either. The application code is plain Vaadin:

```
@Route("")
public class MainView extends VerticalLayout {
    public MainView() {
        TextField name = new TextField("Your name");
        Paragraph greeting = new Paragraph();
        add(new H1("Vidocq + Vaadin"), name,
                new Button("Say hello", e -> greeting.setText("Hello " +
    }
}
```

The full pom and the `module-info` are in the Vaadin guide of the documentation, and VidocqTools/vaadin-on-vidocq is a project ready to clone.

Note the one property at the top, `vidocq.app.layer=all`. We come back to it below, because it is the single setting Vaadin cannot do without.

## Four ways to run it

| What you get | Start |
|---|---|
| `vidocq:dev` The application in a child JVM, reloaded in place on every change | ~330 ms |
| `vidocq:run` The same child JVM, without the reload, as in production | ~310 ms |
| `vidocq:package` A distribution with `lib/`, `app/` and launch scripts, 19 MB without a JDK | ~310 ms |
| `vidocq:jlink` A self-contained image with its own JDK, 68 MB | ~330 ms |

The start times are the runtime's own `Started in` line, measured on an Apple M5 Max with Temurin 25. The figures for the distribution and the image are the median of five starts. The JVM adds about 150 ms around them. All of it was measured in Vaadin's production mode ( `-Dvaadin.productionMode=true` ).

## Development: `vidocq:dev` and `vidocq:run`

```
./mvnw clean package
./mvnw vidocq:dev
```

Build once first: `vidocq:dev` and `vidocq:run` neither build Vaadin's frontend nor process the Vaadin jars, so on an empty `target/` the page answers 404 or 500. The `jvmArgs` properties of the pom start Vaadin in production mode.

This is the everyday loop. The runtime sits on the module path, and your application and its libraries live in a layer above it, which `vidocq:dev` throws away and rebuilds on each change. That is the same in-JVM reload as for any Vidocq application; Vaadin's 69 jars ride along.

## Shipping: `vidocq:package`

```
./mvnw clean package vidocq:package
target/my-app-1.0/bin/my-app.sh
```

You get a folder you can zip and copy anywhere a JDK 25 is installed. The runtime goes in `lib/`, and your application and its libraries go in `app/`. The script starts the runtime and hands it `app/`.

## Self-contained: `vidocq:jlink`

```
./mvnw -Pjlink clean package vidocq:jlink
JDK_JAVA_OPTIONS=-Dvaadin.productionMode=true target/dist/bin/my-app
```

`jlink` builds a trimmed JDK that contains exactly the modules your application needs, with your application inside. The result runs on a machine with no Java installed. The catch is that `jlink` only accepts named Java modules, and not one Vaadin jar is a named module. This is where most of the work went, and why the `jlink` profile adds `vauban:modularize`, whose version the parent manages, and two flags, `vidocq.jlink.mergeSharedResources` and `vauban.modularize.forceExplicit`. All three are explained below; the guide has the profile.

The Vaadin extension of the setup matters here. Vaadin finds your `AppShellConfigurator`, the class that carries a theme or stylesheets, through `ServiceLoader` from `flow-server`, which an image cannot allow: without the extension, the first request never answers. It changes nothing under the other goals. The guide explains why: **An app shell in a jlink image**.

***Note***

**2026-10-25**: this post now shows the 0.4.0 setup. The parent wires the codegen bundles and manages `vauban-maven-plugin`, the application needs no main class, `vidocq-runtime-vaadin-extension` joins the setup, and the project is built once before `vidocq:dev` or `vidocq:run`.

## Why Vaadin needs `vidocq.app.layer=all`

Vidocq starts your application in a module layer of its own, above the runtime's layer. By default, that layer holds your classes only, and your libraries stay with the runtime.

Vaadin does not like that. At startup it looks for your application's resources, such as `META-INF/VAADIN/webapp/index.html`, through the class loader that loaded Vaadin itself. With Vaadin in the lower layer, your resources in the upper layer are out of its sight. Every request then answers 500: *Unable to find index.html*.

`vidocq.app.layer=all` moves every library outside the runtime into your application's layer. A servlet container does the same with `WEB-INF/lib`: the framework and the application share a loader. Set the property once in the pom and every goal uses it.

## What we had to fix on the way

Making this work took about a dozen fixes across Vidocq, Vauban (our CDI container, which also hosts the module tooling) and Foy. A few of them are worth telling, because they show where a class-path framework and a module-path runtime disagree.

### Your dependencies are your code

Vaadin registers itself through a `@WebListener` and a few `ServletContainerInitializer` classes inside `flow-server`. Foy, like most of Vidocq, discovers such components at build time, with annotation processors that run when your code compiles. But `flow-server` is not your code: it was compiled long ago, by Vaadin, without Foy's processors. So nothing found the listener, and every page

The fix is a rule we now apply everywhere: an external dependency is processed like application code. `vidocq:generate` runs every brick's processors over the dependency jars that need them, Foy's included, and the startup report shows it:

```
layer       69 modules, 68 libraries, 2 enriched (Foy 2) (vidocq.app.path,
```

The output is cached, so only the first build pays for it.

## A `uses` directive nobody wrote

A Java module may only call `ServiceLoader` for a service it declares with `uses` in its `module-info`. Vaadin has no `module-info`, so for `jlink` we generate one. Vauban's `vauban:modularize` goal reads the bytecode and works out the `requires`, `provides` and `uses`.

Vaadin taught that analysis three new tricks. `FeatureFlags` passes `getClass().getClassLoader()` to `ServiceLoader`, which our scanner mistook for a second class constant, so it dropped the service type. `Lookup.lookupAll` reaches `ServiceLoader` through an interface, which the scanner did not follow. And `reactor-core`, deep in the dependency tree, ships service files for two optional libraries that are not there. Each one cost a boot failure, and each one now has a test.

## The two flags

The last two problems are not ours to fix. They come from how Vaadin lays out its jars, which the module system does not allow. Each flag works around one of them, and without the flag the build stops with a message that names the jars.

`flow-server` and `flow-react` both ship files in the same directory, `com/vaadin/flow/server/frontend/`. On the class path, the two jars simply overlay. For the module system, though, that directory is a package, and a package belongs to exactly one module. Two modules holding it are a *split package*. At run time the JVM

tolerates it for a directory of plain resources, but `jlink`, which has to file every resource under one module, refuses. `vidocq.jlink.mergeSharedResources` moves the files into one module of the image. Only the copies staged for `jlink` change, and the files involved are frontend templates that only Vaadin's build tooling reads.

The second problem is subtler. `Lookup.lookupAll` lives in `flow-server` and calls `ServiceLoader` on behalf of other jars: `collaboration-engine` and `flow-lit-template` pass it their own service types. `ServiceLoader` checks the module of its caller, so `flow-server` would have to declare `uses` for types defined in modules that themselves depend on `flow-server`. That needs a dependency in both directions, a cycle, and Java modules forbid cycles. No legal `module-info` exists for `flow-server` in such an application. `vauban.modularize.forceExplicit` writes one anyway, without the impossible directives. The image works as long as your application does not use Collaboration Engine or LitTemplate.

### A container that could not see into the image

With all of that in place, the first `jlink` image still failed at startup. Vaadin's initializer received `null` instead of the classes it had asked for. Foy finds those classes by scanning jars, and inside a `jlink` image there are no jars: modules live in the image's own `jrt:` file system, which Foy did not walk. Foy now does, and the image serves its first page.

## What we would like from Vaadin

Both flags exist only because of two layouts in Vaadin. We have collected what only Vaadin can change in **one issue**, before taking it upstream:

- keep each resource directory in a single jar;
- let the module that owns a service type call `ServiceLoader` itself, rather than `Lookup` calling it for everyone;
- stable module names: an `Automatic-Module-Name` in every jar, or better a real `module-info`;

look up the application's resources through the context class loader, which would make `vidocq.app.layer=all` optional.

None of this blocks you today. These are the changes that would make Vaadin a first-class citizen of the module path, for us and for everyone else.

## Try it

Everything here is on the 0.4.0 development line. Use the **snapshot builds** until 0.4.0 is released, and follow the **Vaadin guide**. It has the full pom, the `module-info`, production mode for each goal, and the known limits. So far we have checked production mode on a single view: the page and its bundles. Push, websockets and real applications are next, and we would love to hear how yours behaves.

---

# vaadin # servlet # java-modules # jlink # deep-dive

Share Bluesky



**Yann Blazart**
Senior Tech Lead at SCIAM